

Build a Domain-Specific Resume Intelligence LLM Prototype

Objective

Demonstrate your understanding of how to build a domain-specific LLM instead of simply consuming OpenAI/Gemini APIs.

You may use any open-source model (Llama, Qwen, Mistral, Gemma, Phi, etc.).

Part 1 – Research & Model Selection (20 Marks)

Imagine you have been asked to replace GPT-4 Mini for a Resume Intelligence platform.

Prepare a document (2–4 pages) answering:

- Which open-source model would you choose?
- Why this model?
- Parameter size
- Hardware requirements
- Context window
- License
- Pros & Cons
- Why not choose another popular model?

We want to evaluate your technical reasoning, not the "correct" answer.

Part 2 – Dataset Creation (25 Marks)

We have resume parsing as our target task.

Create a small instruction-tuning dataset (minimum **50 high-quality examples**).

Format each example as:

None

```
{  
  "instruction": "...",  
  "input": "...",  
  "output": {  
    ...  
  }  
}
```

The dataset should cover diverse resume formats, missing fields, varying layouts, and edge cases.

Provide a short explanation of how you would scale this dataset to thousands of examples.

We are evaluating dataset quality, not quantity.

Part 3 – Fine-Tuning or Adaptation (25 Marks)

Choose **one** of the following:

Option A (Preferred)

Fine-tune a small open-source model (e.g., 0.5B–3B parameters) using LoRA/QLoRA on a subset of your dataset.

Include:

- Training configuration
- Hyperparameters
- Training script
- Sample outputs before and after fine-tuning
- Discussion of observed improvements

Option B (If you don't have GPU access)

Design a complete fine-tuning pipeline, including:

- Model choice
- Dataset preparation
- Training strategy
- Hardware requirements

- Evaluation plan
 - Estimated costs
 - Potential risks and mitigation
-

Part 4 – Evaluation Framework (20 Marks)

Design an evaluation methodology for a Resume Intelligence model.

Include:

- What metrics would you use?
- How would you measure parsing accuracy?
- How would you detect hallucinations?
- How would you validate JSON correctness?
- How would you compare two model versions?
- How would you perform regression testing after retraining?

Bonus if you automate any part of the evaluation.

Part 5 – Engineering (10 Marks)

Create a small inference script or API that demonstrates how your chosen model would be used.

The implementation can be minimal, but should include:

- Clean project structure
 - Configuration management
 - Error handling
 - README with setup instructions
-

Deliverables

- GitHub repository
- Research document (PDF/Markdown)
- Dataset

- Training code or pipeline design
 - Evaluation plan
 - README
-

Evaluation Rubric (100 Marks)

Category	Marks
Research & Model Selection	20
Dataset Quality	25
Fine-Tuning / Training Strategy	25
Evaluation Methodology	20
Engineering & Documentation	10